

Deepfake Audio Detection System

Preprocessing · Feature Extraction · Model Architecture

2D CNN on Log-Mel Spectrograms | June 2026

Python 3.x

TensorFlow / Keras

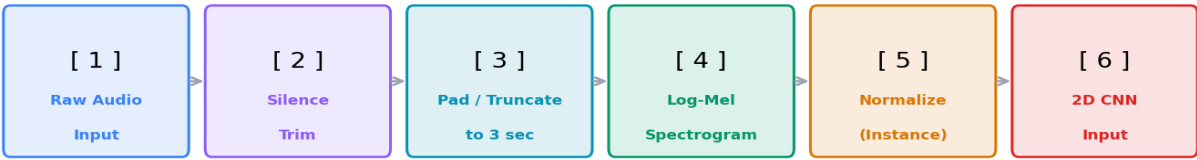
librosa

scikit-learn

kagglehub

01 Preprocessing Pipeline

Transforming raw audio files into clean, uniform-length audio arrays



All audio is passed through a deterministic, reproducible preprocessing pipeline before feature extraction. The pipeline ensures every sample enters the model in a consistent, normalized format regardless of original recording conditions.

1.1 Audio Loading & Standardization

Sample Rate	16,000 Hz — standard for speech processing; balances fidelity vs. compute
Duration	3 seconds — captures sufficient phonetic content for classification
File Format	.wav and .flac supported via librosa.load()
Mono Mixing	Multi-channel audio is automatically downmixed to mono

1.2 Silence Trimming (Inference Only)

In the real-world inference script (predict.py), leading and trailing silence is stripped using librosa.effects.trim() with a threshold of 20 dB. This ensures that microphone recordings with dead air at the start or end do not bias the spectrogram toward silence.

```
# Strip silence before processing (predict.py)
audio, _ = librosa.effects.trim(audio, top_db=20)
```

1.3 Padding & Truncation

All clips are forced to exactly 48,000 samples (16,000 Hz × 3 sec). Short clips are padded; long clips are truncated.

Training padding	mode='constant' — zero-padded silence appended to short clips
Inference padding	mode='wrap' — audio loops instead of zero-padding, avoiding unnatural digital silence in mic recordings
Truncation	audio[:target_length] — clips longer than 3 sec are trimmed from the end

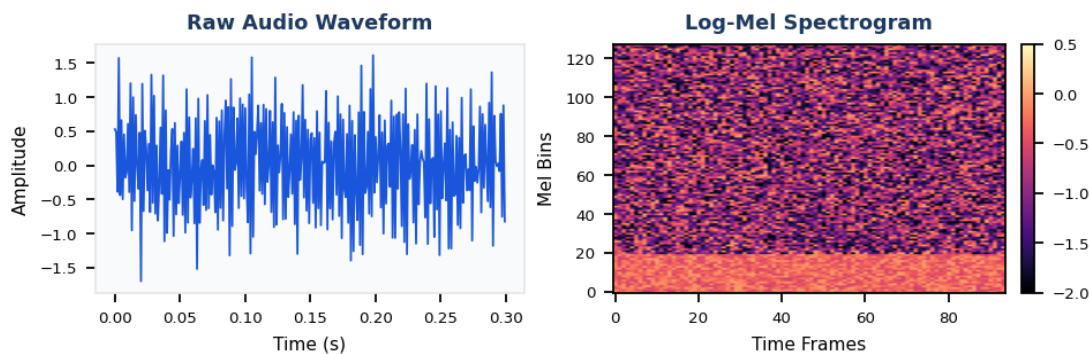
1.4 Dataset Balancing

To avoid class imbalance bias, up to 2,500 samples are randomly drawn from each class (Genuine and Deepfake) using a fixed random seed of 42 for reproducibility. The final balanced dataset is split 80/20 (train/test) with stratified sampling.

02

Feature Extraction

Converting 1D audio waveforms into 2D Log-Mel Spectrograms



Rather than using hand-crafted 1D features like MFCCs, this system converts audio into 2D Log-Mel Spectrograms — treating audio classification as a computer vision problem. This allows the CNN to detect spatial patterns (e.g., phase-distortion artifacts, unnatural harmonic regularities) that are invisible in raw waveforms.

2.1 Mel Spectrogram

Function	librosa.feature.melspectrogram()
Mel Bins (n_mels)	128 — high frequency resolution for detecting subtle artifacts
Max Frequency (fmax)	8,000 Hz — covers the full range of human speech
Output shape	(128, T) where T is the number of time frames (~94 for 3 sec)

2.2 Log Conversion (Power → dB)

The power spectrogram is converted to decibels using `librosa.power_to_db(ref=np.max)`. This log scaling mimics the non-linear perception of human hearing and compresses the dynamic range, making spectral anomalies more visually distinct.

```
mel_spec = librosa.feature.melspectrogram(y=audio, sr=16000, n_mels=128, fmax=8000)
```

```
log_mel_spec = librosa.power_to_db(mel_spec, ref=np.max) # Shape: (128, T)
features = log_mel_spec.T # Transpose → (T, 128)
```

2.3 Normalization

Instance-level normalization is applied to each spectrogram independently, ensuring the model focuses on structural patterns rather than raw volume differences between recordings.

Training	Global mean/std across the full batch: $X = (X - \mu) / (\sigma + \epsilon)$
Inference	Per-instance: $\text{features} = (\text{features} - \text{mean}) / (\text{std} + 1e-8)$
ϵ (epsilon)	1e-8 — prevents division by zero on silent segments

2.4 Final Shape for CNN

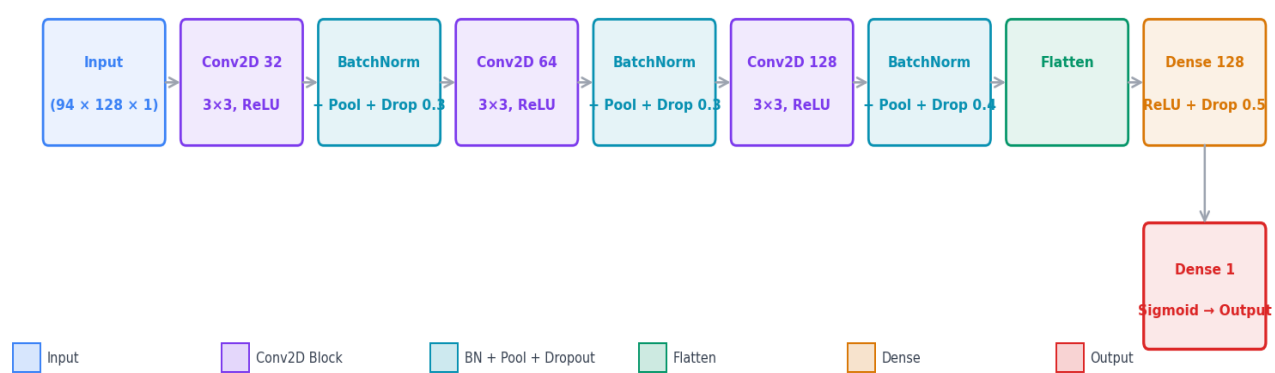
```
# Add channel dimension for 2D CNN (grayscale image format)
features = features[np.newaxis, ..., np.newaxis]

# Final shape: (Batch=1, Time=94, Mels=128, Channel=1)
```

03

Model Architecture

2D Convolutional Neural Network — treating audio as image classification



The model is a sequential 2D CNN with three convolutional blocks of progressively increasing filter depth (32 → 64 → 128), each followed by Batch Normalization, Max Pooling, and Dropout. This hierarchy enables the network to learn features from low-level textures up to high-level phonetic and spectral artifacts.

3.1 Layer-by-Layer Breakdown

Layer	Config	Purpose	Output
Input	Shape: (94, 128, 1)	Log-Mel Spectrogram as grayscale image	94 × 128 × 1
Conv2D	32 filters, 3×3, ReLU	Low-level edge & texture detection	92 × 126 × 32

BatchNorm	—	Stabilize activations per batch	$92 \times 126 \times 32$
MaxPooling2D	2×2	Spatial downsampling	$46 \times 63 \times 32$
Dropout	0.3	Regularization	$46 \times 63 \times 32$
Conv2D	64 filters, 3×3 , ReLU	Mid-level pattern recognition	$44 \times 61 \times 64$
BatchNorm	—	Stabilize activations	$44 \times 61 \times 64$
MaxPooling2D	2×2	Spatial downsampling	$22 \times 30 \times 64$
Dropout	0.3	Regularization	$22 \times 30 \times 64$
Conv2D	128 filters, 3×3 , ReLU	High-level artifact detection	$20 \times 28 \times 128$
BatchNorm	—	Stabilize activations	$20 \times 28 \times 128$
MaxPooling2D	2×2	Spatial downsampling	$10 \times 14 \times 128$
Dropout	0.4	Heavier regularization (deeper layer)	$10 \times 14 \times 128$
Flatten	—	Convert 3D $\rightarrow$ 1D vector	17,920
Dense	128 units, ReLU	Non-linear classification head	128
Dropout	0.5	Heaviest regularization before output	128
Dense	1 unit, Sigmoid	Binary probability output	1 (0–1)

3.2 Design Decisions

2D CNN over 1D	Log-Mel Spectrograms are 2D images — 2D convolutions capture both time-frequency patterns simultaneously
BatchNormalization	Normalizes activations per batch, enabling higher learning rates and faster convergence
Dropout 0.3 $\rightarrow$ 0.5	Progressive dropout increases with depth; heavier regularization at dense layers prevents overfitting
Sigmoid output	Binary classification (0=Genuine, 1=Deepfake); threshold at 0.5
Adam lr=0.001	Adaptive learning rate optimizer; converges reliably on imbalanced spectral data
Binary Crossentropy	Standard loss for binary classification tasks

3.3 Training Configuration

Epochs	35 maximum (with early stopping)
Batch Size	32

Early Stopping	Monitors val_loss with patience=6; restores best weights automatically
Model Checkpoint	Saves best model by val_accuracy to final_deepfake_audio_detector.keras
Validation Split	20% of data held out (stratified split, random_state=42)